GSTR-9 Scrutiny Register — file-by-file reference

Deployed at `~/mayank_malhotra/para-separator-demo/` on `slurm-login01` (`103.42.201.28`).

The system reads a folder of GST case files and produces one Excel workbook with 21 sheets — one per scrutiny parameter — where each row is one company showing the department's defect and the taxpayer's answer to it side by side.

The design rule everything follows: the notice side is deterministic and verbatim; the model is used only where the documents have no structure — the taxpayer's reply — and every word it produces is checked against the source document before it is allowed into a cell.

Contents

```
para-separator-demo/
├── run.sh                  runs the whole pipeline                    78 lines
├── DOCUMENT.md             this file
├── item_desc.xlsx          input data — what each defect alleges      22 rows
├── .gst_api_key            model API key (mode 600)
├── Notices_21-22/          the dataset — 42 GSTINs, 143 PDFs          163 MB
├── pipeline/               all the code — 12 files                 1,932 lines
└── work2/                  generated — caches and intermediate JSON  ~8.4 MB
```

The code, by size:

File	Lines	Role
`pipeline/reply_llm.py`	384	stage 4 — the only stage that judges meaning
`pipeline/scn_split.py`	237	stage 3 — the deterministic split
`pipeline/notice_tables.py`	225	stage 3b — table repair
`pipeline/build2.py`	222	stage 5 — the workbook
`pipeline/params.py`	185	the 21 parameters
`pipeline/text2.py`	146	stage 2 — text and OCR
`pipeline/verify2.py`	145	stage 6 — the checks
`pipeline/vlm_ocr.py`	140	stage 2b — vision OCR
`pipeline/qwen.py`	85	model client
`pipeline/descriptions.py`	75	`item_desc.xlsx` → parameter ids
`pipeline/inventory2.py`	69	stage 1 — file discovery
`pipeline/paths.py`	19	paths

---

1. Top level

`run.sh` — 78 lines

Runs the seven stages in order, each one resumable.

```
bash run.sh          # everything, from PDFs to workbook
bash run.sh build    # skip to the workbook (needs work2/ present)
```

Respects `$PY` so it can be pointed at a specific interpreter, and finds the stage scripts whether it sits beside them or one level above.

It also derives `PATH` from that interpreter and refuses to start if `pdftotext`, `pdftoppm` or `tesseract` are missing. This guard is not decorative. Setting `PY` alone is not enough: `text2.py` and `vlm_ocr.py` shell out to those three binaries, which are resolved from `PATH`, not from the Python environment. Pointing `PY` at a conda environment that contained them — without putting its `bin` on `PATH` — produced a complete run in which **every document extracted to zero characters**, with no error other than a line of `No such file or directory: 'pdftoppm'` scrolling past. The run then cached those empty files, so the damage outlived it.

It now prints what it resolved before doing anything:

```
python   : Python 3.12.13  (/export/home/sobhan/.conda/envs/gst/bin)
pdftotext: /export/home/sobhan/.conda/envs/gst/bin/pdftotext
endpoint : http://slurm-comp01:8033/v1
```

`item_desc.xlsx` — 22 rows, 3 columns

Input data, not output. For each defect: serial number, item name, and a plain-English description of what the department alleges — for example, for `Excess claim of ITC in GSTR-3B w.r.t GSTR-9`:

> *Total ITC claimed in GSTR-3B/GSTR-9 (Table 6A) is higher than the category-wise ITC reported in GSTR-9 (Table 6I), resulting in excess ITC claim.*

These descriptions are what let the model recognise a taxpayer's answer that never names the defect. Without this file the reply stage will not start.

Its rows are numbered 1–22 in the department's order, which is **not** the A/B/C order of the parameter list, and row 16 (interest under s.50(1)) is not one of the 21 parameters at all — hence the explicit mapping in `descriptions.py`.

`.gst_api_key`

The key for the Qwen endpoint, mode 600. `qwen.py` reads it from here, so the key never appears in a command line or in shell history.

`Notices_21-22/Proper order_Adj/<GSTIN>_<suffix>/`

The dataset. 42 case folders, each with three subfolders:

Subfolder	Contents	Used?
`DRC01_SCN/`	the show cause notice (1–2 PDFs)	yes — column 1
`DRC 01 Reply/`	the taxpayer's reply (1–3 PDFs)	yes — column 2
`DRC 01 Order/`	the adjudication order	not yet — column 3 is blank

Measured facts about this corpus, which several design decisions depend on:

- All 42 notices carry native text. **No OCR needed on the notice side.**
- **23 of the 49 reply PDFs are scans** — `pdftotext` returns about one character per page.
- Every notice ends with a `Summary :` recap followed by annexures. Cutting there discards **57% of all notice lines**.

`work2/` — generated, ~8.4 MB

Not shipped code; produced by the run and safe to delete (it costs a re-run).

Path	Written by	Holds
`work2/inventory.json`	`inventory2.py`	every PDF with size and page count
`work2/text/<gstin>/<role>/<file>.txt`	`text2.py`, `vlm_ocr.py`	extracted text, one file per PDF
`work2/text/…/<file>.tess.txt`	`vlm_ocr.py`	tesseract's read, kept as a second witness
`work2/scn.json`	`scn_split.py`	each notice split into its defects
`work2/notice_tables/`, `notice_fixed.json`	`notice_tables.py`	rebuilt tables
`work2/reply_cache/`, `reply.json`	`reply_llm.py`	the taxpayer's answers

---

2. `pipeline/` — the code

Listed in the order the pipeline runs them. The first three are shared infrastructure; the rest are the stages.

Shared

`pipeline/params.py` — 185 lines

The vocabulary everything else is built on.

The 21 scrutiny parameters from `Parameters_TN.pdf`, grouped A (under-declaration of tax, 6), B (excess claim of ITC, 10) and C (interest and late fee, 5). Each carries its official title, its Excel sheet name, and every spelling seen in the corpus — the notices say "Scrutiny of ITC availed under Imports" where the official list says "Excess ITC availed under Imports", and both must match.

`match_heading()` compares on a **squashed** form — lowercased with every non-alphanumeric removed — which absorbs the stray spaces `pdftotext` leaves inside words (`ITC reversal s`, `a n n u a l`) and the punctuation drift between documents. `find_heading()` additionally lets a heading wrap over up to three printed lines, because `pdftotext` breaks it wherever the page did.

Also holds `BOUNDARY`: group headings such as `Excess claim of ITC` and `Late fee calculation`. These get no sheet, but must still be recognised — otherwise the parameter above one of them runs on and swallows it.

`pipeline/paths.py` — 19 lines

Every path in one place: dataset root, work directory, the three role folder names. No stage guesses a location.

`pipeline/qwen.py` — 85 lines

The model client — `Qwen/Qwen3.6-27B-FP8`, thinking disabled, temperature 0.

Three details that are load-bearing rather than cosmetic:

- **Streaming.** Cloudflare fronts the public endpoint and kills a connection that has sent nothing for ~100 seconds. Every long request died with HTTP 524 until the client streamed.
- **Explicit user agent.** The same endpoint 403s Python's default agent.
- **Broad retry.** Under load the HTTP stack raises exceptions that are not `URLError` — a socket dying mid-flush raises `ValueError` — which used to kill a whole worker thread instead of retrying one call.

Reads the key from `GST_API_KEY`, else `.gst_api_key` in the project root.

Stage 1 — `pipeline/inventory2.py` — 69 lines

Walks the 42 case folders and records every PDF with its size and page count.

Chooses the **notice** file by size: the signed DRC-01 is around 1 MB, the portal covering form beside it around 42 KB. Verified correct on all seven cases that have two notice PDFs.

Deliberately does **not** choose the reply file — which reply carries the argument cannot be known before the text exists, because 23 of them are scans that carry nothing at all until OCR has run.

Output: `work2/inventory.json`

Stage 2 — `pipeline/text2.py` — 146 lines

PDF to text, cached per file.

`pdftotext -layout` first — the `-layout` matters, because the department's tables survive only while their column padding is intact.

Falls back to `pdftoppm` + `tesseract` when either test fails:

1. **fewer than 100 characters per page** — the PDF is a photograph of a document, not a document
2. **more than 2% control characters** — the text layer decodes to glyph codes rather than words

The second test exists because of one real file in this corpus: it embeds fonts with a custom encoding and no ToUnicode map, so `pdftotext` returned 39,592 characters containing **4 letters**. It passed the per-page test easily, and Excel refused to store the result at all.

The OCR result is preferred by **letter count, not length** — 39,592 characters of glyph codes beat any honest transcription of the same pages.

Output: `work2/text/<gstin>/<role>/<file>.txt`

Stage 2b — `pipeline/vlm_ocr.py` — 140 lines

Re-reads the scanned PDFs using Qwen's image input, one 150 dpi page at a time.

tesseract recovers the words but destroys the tables, and a GST reply is mostly tables. On one measured page tesseract returned

```
ineligible MCdeciared  —{e [ Te
Excess ITC claimed (1–2) | ssi  4,83,145  4,83,145 | – | 3,82,297 | 13,48,587
```

and pulled the rubber stamp into the body as `KE SOFD, u–| CHENNAI }–5]`. The model returned the same table as clean rows, with no stamp.

tesseract's output is not discarded. It is kept beside the new file as `<name>.tess.txt` and used by `verify2.py` as an independent second witness: a figure that appears in neither read of a scanned page is a figure nobody photographed.

Measured: 23 files, 245 pages, ~13 minutes.

Stage 3 — `pipeline/scn_split.py` — 237 lines

Splits the notice on the 21 headings. No model is involved.

The department prints the parameter headings verbatim in the DRC-01, so this needs only the heading matcher: a hit opens a section, the next heading closes it. Every cell is a literal slice of the cached text, so the notice column is **verbatim by construction** — there is no generation step that could go wrong.

Two things are deliberately discarded:

- **The Summary block.** Every notice ends with `Summary :`, a total-tax table and the "it is proposed to assess" paragraph — a recap of all defects at once, belonging to no single parameter. In the largest case it sits at line 366 of 1,709, and everything below it is annexures. Across the corpus this drops 57% of all notice lines. Two notices carry no Summary line and fall back to the closing paragraph.
- **Group headings** (`Excess claim of ITC`, `Late fee calculation`) — recognised so they can end the section above them, but given no sheet.

Also extracts the trade name, which needs four different rules because this corpus uses four header layouts: letter style (`Name : Tvl. X`), label and value on one line, label above value, value above label, and one where the name appears only inside the subject line.

Output: `work2/scn.json` — 242 defect sections across 42 notices, 5.8 per case.

Stage 3b — `pipeline/notice_tables.py` — 225 lines

Repairs tables that `pdftotext` broke across lines.

The department's tables are wider than the page, so a cell wraps and one figure arrives as two:

```
      GSTR-3B(Table 4A of        1277365 1 1277365 1 1219547
1     GSTR-3B/           6A                                  0 0 124509451
                                                 209                    1
```

which is `12773651`, `12773651`, `1219547209` and `1245094511`. **87 of the 242 sections** have this damage. Nothing textual can reassemble it reliably, because the join is a column position, not a delimiter.

So the section goes to the model to be returned as prose plus real table rows, and the result is checked before use: **every figure must be reconstructible from that section's own digits** — either as a contiguous run, or as two runs joined, which is exactly what a wrapped cell is. A section whose repair cannot be proved keeps its raw verbatim text.

Result: 85 repaired, **81 verified and used**, 4 kept raw.

Output: `work2/notice_fixed.json`

Stage 4 shared — `pipeline/descriptions.py` — 75 lines

Loads `item_desc.xlsx` and maps its 22 rows onto the 21 parameters.

The mapping is **explicit, not positional** — the spreadsheet is in the department's order rather than A/B/C order, and its row 16 has no sheet. It is asserted against `params.py` on import, so a wrong id fails

loudly instead of silently mis-describing an entire sheet.

`brief(pid)` returns the title plus the description, which is what the reply prompt shows the model.

Stage 4 — `pipeline/reply_llm.py` — 384 lines

Finds the taxpayer's answer to each defect the notice raised. The only stage where the model decides anything about meaning.

Keyword matching cannot work here. Taxpayers reproduce a heading when they feel like it and otherwise write "Query No:2", or nothing at all.

Choosing which reply file to read. Ranked by how much a file *argues* (reply phrases per thousand characters), not by size. One case has a 289-page `Annexure A to G` whose digit ratio is low — ledger rows are mostly words — so it wins on length every time and contains no argument at all. Each document is capped at 1,200 lines: a ninety-page reply is four pages of argument followed by its evidence, and paying the model to read the evidence buys nothing but latency. This one change cut the stage from 140 model calls to 63.

Four guards keep the answers honest:

1. **Id fencing.** The model is offered only the parameters that case's own notice raised, and an id outside that list is dropped **by the parser**, not merely forbidden in the prompt. A reply can never invent a defect the department never raised.
2. **Figure check.** Every 4-to-12-digit figure the model writes must exist in the source document, compared on bare digits because the source wraps long cells mid-number.
3. **Grounding check.** Runs of five words in the answer must appear in the reply document — at least 55% of them. This exists *because* the model is now given descriptions: the tempting failure is to paraphrase a description into a reply the taxpayer never filed, and no figure check would catch that. Prose lifted from the document scores near 1.0; prose composed by the model scores near 0.
4. **Line spans.** Every answer also reports the lines it came from.

The failure ladder, which is what makes the column trustworthy:

Situation	What the cell shows
passes both checks	the model's tidied wording
fails a check, has a line span	that **verbatim slice** of the reply
fails a check, no usable span	`No reply`
defect never returned	`No reply`

Ungrounded prose cannot reach the workbook. The worst case is verbatim text, and the fallback of the fallback is silence.

Output: `work2/reply.json` — 102 answers found, 140 `No reply`, 2 fell back to verbatim.

Stage 5 — `pipeline/build2.py` — 222 lines

Writes the workbook.

- A **Contents** sheet: every parameter with its row count, replies and `No reply` count.
- **21 sheets**, one per parameter — including the two that no notice raised, which carry a note. An empty sheet says "not raised"; a missing sheet looks like a pipeline that lost it.
- Columns: `GSTIN | Trade name | Notice (SCN) defect | Taxpayer reply | Officer finding`.

The notice is the **spine**: a company appears on a sheet only if its own DRC-01 raised that parameter, so no row can exist without a defect.

Menlo 9pt on the three text columns — the notice tables hold their shape only while the column padding is preserved, and a proportional font destroys it. Model-returned tables are re-padded the same way so both look alike. Control characters are stripped, since a single one aborts the entire save.

Output: `register_21-22.xlsx`

Stage 6 — `pipeline/verify2.py` — 145 lines

Refuses to bless the workbook unless every one of these holds. Each check exists because that class of mistake has already been made once.

Check	What it prevents
notice cells verbatim	a notice cell that is not a literal run of lines from that case's own PDF
nothing from below the Summary line	the recap or forty pages of annexure leaking into a defect cell
no empty notice cell	a row with nothing in the spine column
only the 21 parameters	a group heading or stray id becoming a sheet
every repaired table is figure-checked	an unproved table repair being shown
every reply cell grounded	model prose that passed neither check reaching a cell
no reply for a defect never raised	id fencing having failed silently
scanned-page figures seen by tesseract too	a figure invented during vision OCR
coverage floors	a partial re-run silently producing a smaller workbook that still looks fine

Exits non-zero on any failure.

---

3. Running it

The environment

The login node ships no poppler and no tesseract, so they live in a dedicated conda environment. **Created and verified:**

```
/export/home/sobhan/.conda/envs/gst
  python      3.12.13
  openpyxl    3.1.5
  pdftotext   26.07.0     (poppler)
  pdftoppm    26.07.0     (poppler)
  tesseract   5.5.3
```

If it ever needs rebuilding:

```
conda create -y -n gst -c conda-forge python=3.12 poppler tesseract openpyxl
```

Note the path — `~/.conda/envs/gst`, the user's conda directory, **not** `~/mayank_malhotra/miniconda/envs/`, which is where the system conda base lives.

The command

```
cd ~/mayank_malhotra/para-separator-demo
export GST_API_URL=http://slurm-comp01:8033/v1
PY=~/.conda/envs/gst/bin/python bash run.sh
```

The key is read from `.gst_api_key`, so nothing secret is typed. Note vLLM listens on **slurm-comp01**, not the login node — nothing answers on `localhost:8033` there.

Equivalently, activating the environment sets `PATH` itself:

```
conda activate gst
export GST_API_URL=http://slurm-comp01:8033/v1
bash run.sh
```

Full run from the PDFs is roughly 35 minutes, dominated by OCR. With `work2/` already present, `bash run.sh build` produces the workbook in seconds.

To demonstrate a single stage — this one uses no model at all, and is the clearest thing to show:

```
cd pipeline
python scn_split.py 33AAACC0460H1Z9
```